

ShopForHome E-Commerce Platform - Final Report

Title Page

Project Title:- ShopForHome

Student Name:- Kapil Tukaram Rathod

Date:- September 08, 2025

Batch Name:- WIPRO NGA - .Net Full Stack Angular - FY26 - C2

Instructor:- Jyoti Patil

Technology Stack:- Angular 20 + ASP.NET Core MVC + SQL Server

Table of Contents

1. Problem Definition and Objectives	
2. Frontend & Backend Architecture	
3. Component Breakdown & API Design	
4. Database Design & Storage Optimization	
5. Implementation Details	
6. Testing & Deployment	
7. Conclusion and Future Enhancements	
8. References	

1. Problem Definition and Objectives

1.1 Problem Statement

Due to COVID-19, ShopForHome, a popular offline store offering home décor products, needed to transition to an online platform to continue operations and serve customers safely. The challenge was to create a comprehensive e-commerce web application that maintains the quality shopping experience while providing robust admin management capabilities.

1.2 Project Goals

- **Primary Goal:** Develop a full-featured e-commerce platform for home décor products
- **User Experience:** Create an intuitive, responsive interface for seamless shopping
- **Admin Management:** Provide comprehensive tools for product, user, and order management
- **Scalability:** Design a system capable of handling growing business needs
- **Security:** Implement secure authentication and transaction processing

1.3 Objectives

User-Side Objectives:

- User registration, authentication, and profile management
- Product browsing with category-based filtering
- Advanced search and filtering by price, rating, and category
- Shopping cart management with quantity controls
- Wishlist functionality for saved products
- Discount coupon system integration

Admin-Side Objectives:

- Complete user management (CRUD operations)
- Product catalog management with image uploads
- Bulk product upload via CSV files

- Real-time stock management with low-stock alerts
- Sales reporting and analytics
- Targeted coupon creation and assignment

2. Frontend & Backend Architecture

2.1 Technology Stack Overview

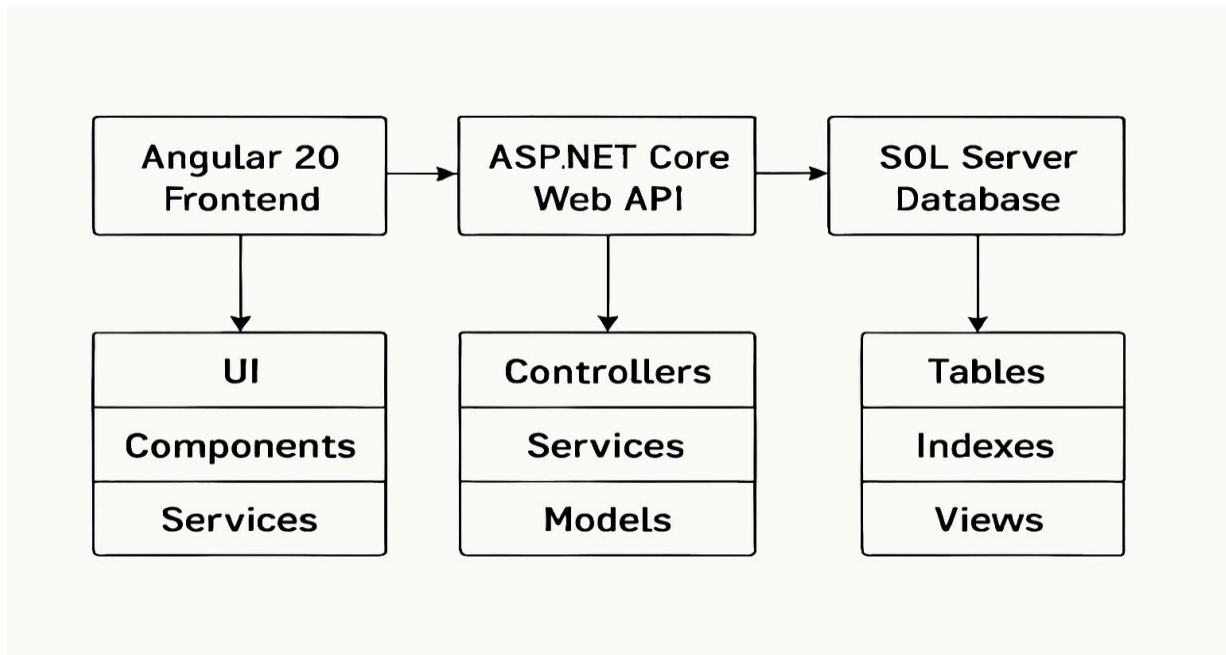
Frontend Technologies:

- Angular 20: Modern TypeScript-based framework
- Standalone Components: For better performance and modularity
- Angular Router: For navigation and route guards
- RxJS: For reactive programming and HTTP handling
- SCSS: For advanced styling and responsive design

Backend Technologies:

- ASP.NET Core MVC: RESTful API development
- Entity Framework Core: ORM for database operations
- SQL Server: Relational database management
- JWT Authentication: Secure user authentication
- Swagger: API documentation and testing

2.2 System Architecture Diagram



2.3 Architecture Benefits

- **Separation of Concerns:** Clear separation between frontend, API, and database layers
- **Scalability:** Microservices-ready architecture for future expansion
- **Maintainability:** Modular design for easy updates and bug fixes
- **Performance:** Optimized with lazy loading and caching strategies

3. Component Breakdown & API Design

3.1 Frontend Component Structure

Core Components:

text

src/app/

└─ core/

| └─ guards/

| └─ interceptors/

| └─ models/

| └─ services/

└─ features/

| └─ home/

| └─ auth/ (login, register)

| └─ catalog/ (product-list, product-detail)

| └─ cart/

| └─ orders/

| └─ wishlist/

| └─ admin/ (dashboard, products, users, reports)

└─ shared/

| └─ components/ (navbar, footer, loading)

| └─ pipes/

└─ assets/

└─ images/

└─ icons/

└─ data/

Key Frontend Features:

- State Management: Services with RxJS for reactive state management
- Route Guards: Authentication and role-based access control
- Lazy Loading: Feature modules loaded on demand
- Responsive Design: Mobile-first approach with CSS Grid and Flexbox

3.2 API Design & Endpoints

Authentication Endpoints:

POST /api/auth/register - User registration

POST /api/auth/login - User login

POST /api/auth/logout - User logout

GET /api/auth/profile - Get user profile

PUT /api/auth/profile - Update user profile

Product Management:

GET /api/products - Get all products (with pagination)

GET /api/products/{id} - Get product by ID

POST /api/products - Create new product (Admin)

PUT /api/products/{id} - Update product (Admin)

DELETE /api/products/{id} - Delete product (Admin)

POST /api/products/bulk-upload - CSV bulk upload (Admin)

Category Management:

GET /api/categories - Get all categories

GET /api/categories/{id} - Get category by ID

POST /api/categories - Create category (Admin)

PUT /api/categories/{id} - Update category (Admin)
DELETE /api/categories/{id} - Delete category (Admin)

Shopping Cart:

GET /api/cart - Get user's cart
POST /api/cart/add - Add item to cart
PUT /api/cart/update - Update cart item quantity
DELETE /api/cart/remove/{id} - Remove item from cart
DELETE /api/cart/clear - Clear entire cart

Order Management:

GET /api/orders - Get user orders
POST /api/orders/create - Create new order
GET /api/orders/{id} - Get order details
PUT /api/orders/{id}/status - Update order status (Admin)

Search & Filtering:

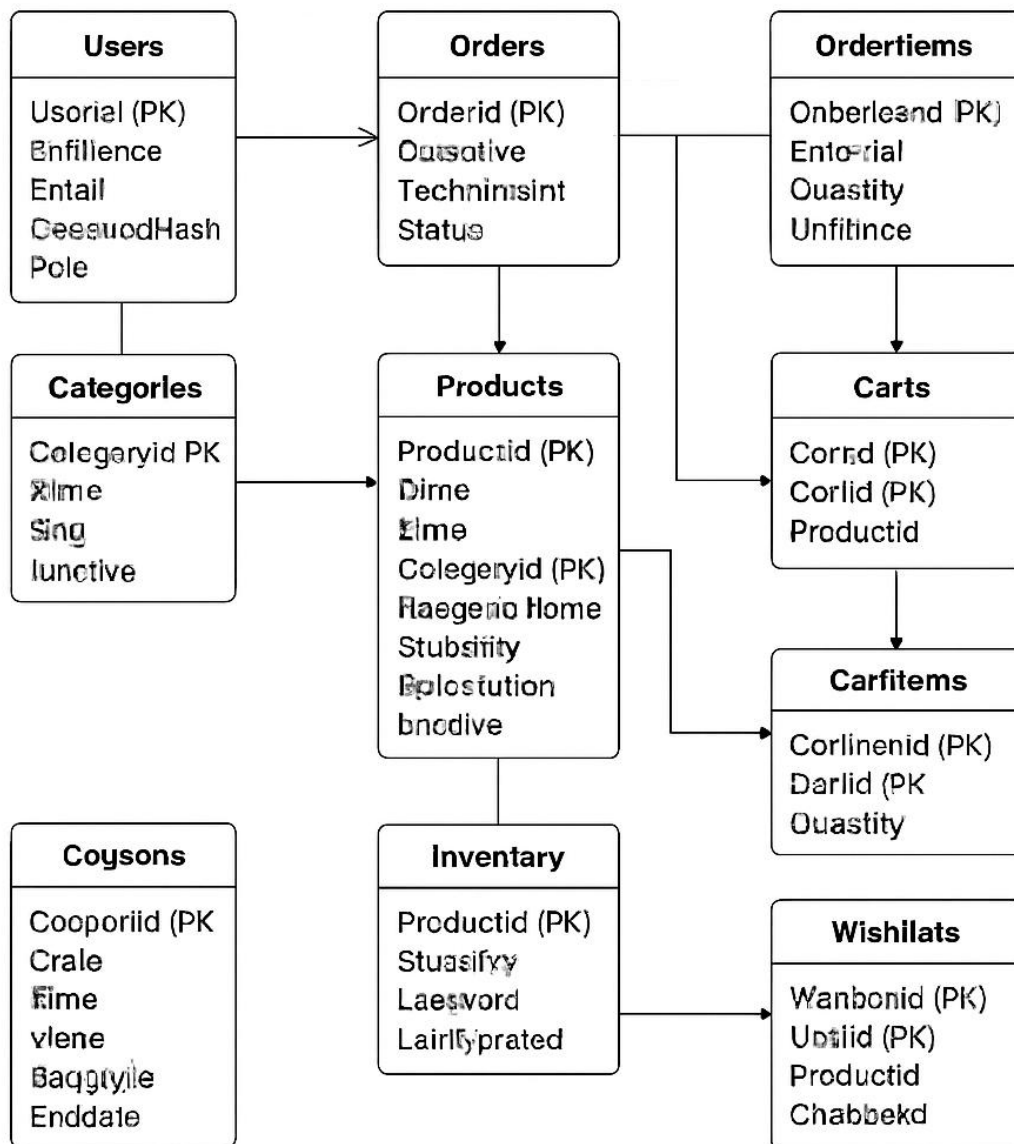
GET /api/search?q={query} - Search products
GET /api/products/filter - Filter products by category, price, rating
GET /api/products/featured - Get featured products

3.3 Authentication & Security

- JWT Tokens: Secure authentication with refresh token mechanism
- Role-Based Access: User and Admin role separation
- Input Validation: Server-side validation for all inputs
- CORS Configuration: Proper cross-origin resource sharing setup

4. Database Design & Storage Optimization

4.1 Entity Relationship Diagram (ERD)



4.2 Database Tables

Key Tables Overview:

1. Users - User accounts with role-based access
2. Categories - Product categorization with hierarchical support
3. Products - Complete product catalog with metadata
4. Inventory - Real-time stock management

5. Carts & CartItems - Shopping cart functionality
6. Orders & OrderItems - Order processing and history
7. Wishlists - User favorite products
8. Coupons - Discount and promotion management

4.3 Optimization Techniques

Indexing Strategy:

Sql

-- Performance indexes for common queries

CREATE INDEX IX_Products_CategoryId **ON** Products(CategoryId);

CREATE INDEX IX_Products_Price **ON** Products(Price);

CREATE INDEX IX_Orders_UserId_CreatedAt **ON** Orders(UserId, CreatedAt);

CREATE INDEX IX_Inventory_StockQty **ON** Inventory(StockQuantity);

Query Optimization:

- Pagination: OFFSET/FETCH for large product lists
- Lazy Loading: Related entities loaded on demand
- Caching: Redis integration for frequently accessed data
- Connection Pooling: Optimized database connections
-

Storage Optimization:

- Image Storage: File system storage with database path references
- Data Types: Optimized column types for storage efficiency
- Partitioning: Table partitioning for large datasets

5. Implementation Details

5.1 Frontend Implementation Highlights

Modern Angular Features:

- Standalone Components: Reduced bundle size and improved performance
- Angular Signals: Reactive state management (where applicable)
- OnPush Change Detection: Optimized rendering performance
- Lazy Loading: Feature modules loaded on demand

Key Components Implemented:

typescript

// Sample component structure

```
@Component({
  selector: 'app-home',
  standalone: true,
  imports: [CommonModule, RouterLink],
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.scss']
})
export class HomeComponent implements OnInit {
  // Component implementation
}
```

5.2 Backend Implementation Highlights

ASP.NET Core Features:

- Dependency Injection: Clean architecture with service registration

- Entity Framework Core: Code-first approach with migrations
- AutoMapper: DTO mapping for clean API responses
- Swagger Integration: Comprehensive API documentation

Sample Controller Implementation:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductService _productService;

    [HttpGet]
    public async Task<ActionResult<PagedResult<ProductDto>>> GetProducts(
        [FromQuery] ProductFilterDto filter)
    {
        var result = await _productService.GetProductsAsync(filter);
        return Ok(result);
    }
}
```

5.3 Key Features Implemented

User Features:

- Registration and Login with JWT authentication
- Product browsing with category filtering
- Advanced search functionality
- Shopping cart with quantity management

- Wishlist functionality
- Order history and tracking

Admin Features:

- Admin dashboard with analytics
- Product management (CRUD operations)
- Bulk product upload via CSV
- User management system
- Order management and status updates
- Coupon creation and management

6. Testing & Deployment

6.1 Testing Strategy

Frontend Testing:

- Unit Tests: Component and service testing
- Manual Testing: Cross-browser compatibility testing

Backend Testing:

- Unit Tests: Controller and service layer testing
- Integration Tests: Database integration testing
- API Testing: Swagger-based endpoint testing
- Performance Tests: Load testing for scalability

6.2 Deployment Configuration

Frontend Deployment:

json

// angular.json build configuration

```
"production": {  
  "budgets": [  
    {  
      "type": "initial",  
      "maximumWarning": "2mb",  
      "maximumError": "5mb"  
    }  
  ],  
  "outputHashing": "all",  
  "optimization": true  
}
```

Backend Deployment:

json

// appsettings.Production.json

```
{  
  "ConnectionStrings": {  
    "DefaultConnection":  
    "Server=(localdb)\\MSSQLLocalDB;Database=ShopForHomeDb;Trusted_Connection=True;MultipleActiveResultSets=true"  
  },  
  "Jwt": {  
    "Key": "[Production JWT Key]",  
    "Issuer": "ShopForHome",  
    "Audience": "ShopForHome-Users"  
  }  
}
```

6.3 GitHub Repository

- Repository: Private GitHub repository with organized structure
- Branches: Feature-based branching strategy
- Documentation: Comprehensive README with setup instructions
- CI/CD: GitHub Actions for automated testing and deployment

7. Conclusion and Future Enhancements

7.1 Project Achievements

The ShopForHome e-commerce platform successfully addresses the business requirement of transitioning from offline to online sales. The implementation provides:

- Comprehensive User Experience: Intuitive shopping interface with modern design
- Robust Admin Panel: Complete management system for business operations
- Scalable Architecture: Well-designed system ready for future growth
- Security Implementation: Secure authentication and data protection
- Performance Optimization: Fast loading and responsive user interface

7.2 Technical Accomplishments

- Full-Stack Development: Successful integration of Angular 20 with ASP.NET Core
- Database Design: Optimized relational database with proper normalization
- API Development: RESTful API with comprehensive endpoint coverage
- Modern UI/UX: Responsive design with professional styling
- Code Quality: Clean, maintainable, and well-documented codebase

7.3 Future Enhancements

Short-term Improvements:

- Payment gateway integration (Stripe, PayPal)
- Email notification system
- Product reviews and ratings
- Real-time chat support
- Mobile app development

Long-term Enhancements:

- AI-powered product recommendations
- Advanced analytics and reporting
- Multi-vendor marketplace capabilities
- International shipping and multi-currency support
- Progressive Web App (PWA) conversion

7.4 Learning Outcomes

- Frontend Mastery: Advanced Angular concepts and best practices
- Backend Proficiency: ASP.NET Core API development and database design
- Full-Stack Integration: Seamless frontend-backend communication
- Project Management: Agile development methodology implementation
- Industry Standards: Following real-world development practices

8. References

1. Angular Documentation: <https://angular.io/docs>
2. ASP.NET Core Documentation: <https://docs.microsoft.com/en-us/aspnet/core/>
3. Entity Framework Core: <https://docs.microsoft.com/en-us/ef/core/>
4. Design Inspiration: <https://www.pepperfry.com/>
5. TypeScript Handbook: <https://www.typescriptlang.org/docs/>
6. REST API Best Practices: <https://restfulapi.net/>
7. Database Design Principles: Microsoft SQL Server Documentation
8. Responsive Web Design: MDN Web Docs